Systematic Literature Review Report

Alla Krylova 196722
Oleksandr Nychyporchuk 196659
Kiryl Pashkevich 196687
Pavel Khmialeuski 197055

1 Research project

1.1 Title

The impact of service-mesh solutions on the efficiency of microservice applications

1.2 Supervisor

Krzysztof Gierłowski (KT)

1.3 Goals and short description

Currently, a significant portion of applications deployed in cloud environments utilize microservices architecture. This type of architecture requires decomposing application functionality into component elements, implemented by individual microservices, and then deploying them in an environment that allows the microservices to communicate efficiently and reliably. This task is most often accomplished by orchestration platforms such as Kubernetes. Maintaining the separation and security of applications sharing the same deployment platform is also a key aspect of this type of deployment. Service mesh solutions are often used for this purpose, as they can automatically create secure communication environments for specific applications.

The aim of this project is to analyze and experimentally test the impact of using service mesh solutions on the efficiency (and particularly performance) of an application composed of multiple microservices.

2 Systemic Literature Review plan

2.1 Goals and questions:

The goal of this systematic literature review is to identify, categorize, and synthesize existing research on service mesh technologies in the context of microservices applications, with a particular focus on comparing environments with and without service mesh in terms of performance implications, security benefits, and deployment trade-offs.

1. What are the main service mesh implementations used in production microservice systems?
2. What performance metrics (latency, throughput, resource consumption) are most often evaluated when comparing service mesh vs. no service mesh deployments?
3. What security mechanisms (mTLS, authorization, policies) do service meshes provide, and what is their performance overhead compared to native Kubernetes security?
4. What architectural deployment patterns and migration strategies are described for adopting service mesh in existing microservice applications?
5. How do monitoring and observability capabilities differ between service mesh and non-service mesh environments?

2.2 Keywords:

1. Core concept: “service mesh”, “service meshes”
2. Comparison focus: “without service mesh”, “standard Kubernetes”, baseline, “vs native”

3. Specific implementations: Istio, Linkerd, Consul
4. Related technologies: “microservices architecture”, “micro-service*”, “micro services”, Kubernetes
5. Performance aspects: performance, latency, throughput, overhead, scalability, efficiency

2.3 Search strings:

Base search string:
(“service mesh” OR “service meshes” OR istio OR linkerd OR consul OR kuma OR “aws app mesh” OR “open service mesh” OR osm OR cilium)
AND (microservice* OR “micro-service*” OR “micro services” OR “microservices architecture” OR kubernetes)
AND (performance OR latency OR throughput OR overhead OR scalability OR efficiency OR comparison OR “vs native” OR baseline OR “without service mesh” OR “standard kubernetes”)

2.4 Literature databases:

1. Scopus
2. SpringerLink
3. Arxiv
4. IEEE Xplore

These 4 databases provide comprehensive coverage of technical literature in computer science, software engineering, and distributed systems, with minimal overlap and complementary strengths.

2.5 Inclusion criteria:

1. Publication type: peer-reviewed journal articles and conference proceedings.
2. Language: English.
3. Years: 2017-2025 (Service Mesh has been actively developing since 2017, when Istio was released). Additionally, Linkerd 1 is an important topic. It is actually the first service mesh solution, but much more distanced from modern days concept of service mesh.
4. Availability: Full text available.

2.6 Exclusion criteria:

1. Article length: Short articles (less than 4 pages)
2. Relevance: Articles that mention Service Mesh only in passing or as a secondary topic
3. Language: Articles not in English

2.7 Quality criteria:

Each article will be assessed against the following criteria (scored 0 or 1 point per criterion). Articles must score at least 4 out of 7 points to proceed to data extraction.

1. Clarity of description: Is the architecture, experimental setup, or methodology clearly described?
2. Quantitative evaluation: Does the article provide quantitative performance measurements (latency, CPU, memory, throughput)?
3. Baseline comparison: Does the article compare service mesh performance against a non-service-mesh and/or compare different service mesh solutions baseline (native Kubernetes, no sidecar, etc.)?
4. Experimental rigor: Are testing conditions (workload, environment, hardware/software specifications) clearly described?
5. Statistical validation: Are results validated (statistical significance, confidence intervals, reproducibility)?
6. Security analysis: Does the article provide security analysis or vulnerability assessment?
7. Publication venue: Is the article published in a reputable venue?

2.8 Data extraction:

The following data will be extracted from each selected article and recorded in a structured spreadsheet (Google Sheets).

1. Article metadata:
   a) Article ID: Unique identifier
   b) Authors: Full list of authors
   c) Year: Publication year
   d) Title: Full article title
   e) Source: Journal/conference name
   f) DOI: Digital Object Identifier
2. Research context
   a) Service Mesh type: Specific implementations studied (Istio, Linkerd, Consul, etc.)
   b) Baseline used: What "no service mesh" baseline was used? (Native Kubernetes, no sidecar, etc.)
   c) Deployment context: Cloud, on-premise, hybrid, edge
   d) Orchestration platform: Kubernetes, Nomad, etc.
   e) Application type: Example application used (if any)
3. Methodology
   a) Research method: Experiment, case study, survey, review, simulation
   b) Scale: Number of services, nodes, requests
   c) Workload characteristics: Type of workload, request patterns
4. Performance metrics
   a) Latency: Measured values (mean, p95, p99)
   b) Throughput: Requests per second, data transfer rate
   c) CPU usage: Measured consumption
   d) Memory usage: Measured consumption
   e) Network overhead: Additional bytes transferred
5. Security aspects
   a) Security mechanisms: mTLS, RBAC, policies, authentication
   b) Security impact: Performance cost of security features
6. Observability
   a) Monitoring tools: Prometheus, Grafana, Kiali, Jaeger, Zipkin
   b) Observability features: Metrics, logs, traces, visualization
7. Results
   a) Key findings: Main conclusions of the study
   b) Limitations: Stated limitations of the research
   c) Future work: Suggested research directions

2.9 SLR process:

Step	Description	Executor	Verifier	Tools
Step 1	Database search execution	Oleksandr Nychyporchuk	Kiryl Pashkevich	Scopus, ACM, Web of Science
Step 2	Export results and import to reference manager	Oleksandr Nychyporchuk	Pavel Khmialeuski	Zotero/Mendeley
Step 3	Duplicate removal	Alla Krylova	Pavel Khmialeuski	Zotero, Excel

Step 4	Title and abstract screening (Round 1)	Alla Krylova, Pavel Khmialeuski	Oleksandr Nychyporchuk, Kiryl Pashkevich	Google Sheets
Step 5	Full-text retrieval	Kiryl Pashkevich	Alla Krylova	University library access, DOI resolvers
Step 6	Full-text screening (Round 2) and quality assessment	Kiryl Pashkevich, Oleksandr Nychyporchuk	Alla Krylova, Pavel Khmialeuski	Google Sheets, PDF readers
Step 7	Snowballing (forward and backward)	Pavel Khmialeuski	Alla Krylova	Google Scholar, reference lists
Step 8	Data extraction	All team members (divided by articles)	Oleksandr Nychyporchuk, Kiryl Pashkevich	Google Sheets template
Step 9	Synthesis and reporting	Kiryl Pashkevich	Alla Krylova	Google Docs, LaTeX

3 Systematic Literature Review results

3.1 Results in numbers

Number of articles retrieved from databases (before deduplication):

- Scopus: 75
- SpringerLink: 5
- Arxiv: 28
- IEEE Xplore: 179

3.2 Articles selected for data extraction

The following list presents the 12 articles selected after verification and qualified for data extraction:

[1] “Elastic Scaling of Real-Time Communication Services”
[2] “Trade-Offs in Kubernetes Security and Energy Consumption”
[3] “Network shortcut in data plane of service mesh with eBPF”
[4] “Evaluation of a Smart Intercom Microservice System Based on the Cloud of Things”
[5] “Resilient microservices: an investigation into Istio effectiveness in Kubernetes”
[6] “Technical Report: Performance Comparison of Service Mesh Frameworks: the MTLS Test Case”
[7] “Impact of etcd deployment on Kubernetes, Istio, and application performance”
[8] “Challenges and Opportunities in Performance Benchmarking of Service Mesh for the Edge”
[9] “Security Issues and Challenges in Service Meshes – An Extended Study”
[10] “Full-stack vulnerability analysis of the cloud-native platform”
[11] “Comparative Evaluation of Linkerd and Istio Service Meshes in a Microservices Architecture Application”
[12] “Microarchitectural Analysis and Characterization of Performance Overheads in Service Meshes with Kubernetes”

3.3 Initial extracted data

Main focus / Keywords	Service Mesh	Article
Edge computing, performance benchmarking, latency, throughput, Iptables overhead	Istio (Envoy)	[8]
Title: Challenges and Opportunities in Performance Benchmarking of Service Mesh for the Edge **Summary:** This paper investigates the architectural complexities and performance impacts of deploying a service mesh in latency-sensitive edge environments. By benchmarking north-south and east-west communications, the authors identify significant bottlenecks in the Linux network stack. Specifically, they highlight that Linux's Iptables rule matching (used heavily by sidecar proxies) creates substantial CPU micro-architecture overhead at scale. This emphasizes the need for optimized data planes when deploying service meshes at the edge.		
Chaos engineering, resilience, fault tolerance, response time, heavy load	Istio	[5]
Title: Resilient microservices: an investigation into Istio effectiveness in Kubernetes **Summary:** This study evaluates the impact of the Istio service mesh on the resilience of Kubernetes clusters using chaos engineering principles (injecting failures into the system). Performance metrics such as response time, error rates, and resource usage were analyzed under increased load. The findings demonstrate that Istio significantly improves system stability, failure resilience, and recovery times compared to a traditional, non-mesh Kubernetes architecture.		
mTLS overhead, latency, memory consumption, sidecar vs. sidecar-less	Istio, Linkerd, Cilium	[6]
Title: Technical Report: Performance Comparison of Service Mesh Frameworks: the MTLS Test Case **Summary:** Recognizing that security is a primary driver for service mesh adoption, this technical report thoroughly evaluates the performance overhead of mutual TLS (mTLS). It compares traditional sidecar architectures (Istio, Linkerd) with sidecar-less and eBPF-accelerated architectures (Istio Ambient, Cilium). The experiments reveal significant differences in latency and memory consumption, noting that while some meshes appear faster, the overhead is heavily dependent on the default security features and the underlying proxy architecture.		
CPU consumption, RAM usage, latency, high-load stability, SEMMA methodology	Istio, Linkerd	[11]
Title: Comparative Evaluation of Linkerd and Istio Service Meshes in a Microservices Architecture Application **Summary:** This paper conducts a direct, objective comparison between Istio and Linkerd using the Online Boutique microservices application under low, medium, and high load scenarios. Experimental results showed that Linkerd significantly outperformed Istio in efficiency, maintaining lower average latency (104 ms vs. 135 ms), consuming less CPU and RAM, and recording zero errors under high load. Conversely, Istio exhibited consumption peaks and internal failures under stress, though the authors note it remains preferable for environments requiring highly advanced control features.		

3.4 Article statistics

To understand the research landscape surrounding service mesh performance in microservice architectures, a quantitative analysis was performed on the search results.

3.4.1 Initial Search Distribution

The initial database queries returned a total of 287 articles before deduplication. As shown in Figure 1, IEEE Xplore (179 articles) and Scopus (75 articles) provided the vast majority of the results. This reflects the topic's strong roots in applied computer science, network engineering, and distributed systems. arXiv (28 articles) and SpringerLink (5 articles) yielded fewer direct matches based on our highly specific search strings.

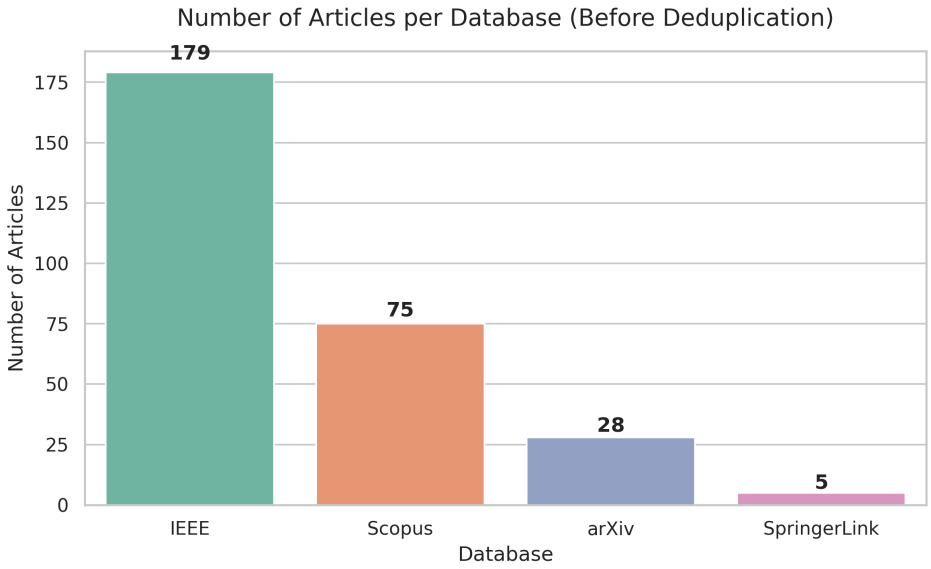

Figure 1: Number of articles retrieved per database before deduplication.

3.4.2 Publication Trends Over Time

After removing duplicates and cleaning the dataset, the temporal distribution of the unique articles was analyzed (Figure 2). The data indicates that while modern service meshes (like Istio) were introduced around 2017, rigorous academic research and performance benchmarking began to gain significant traction from 2020 onward. The steady volume of recent publications demonstrates that service mesh efficiency, security overhead, and edge deployment are currently highly active and maturing research areas.

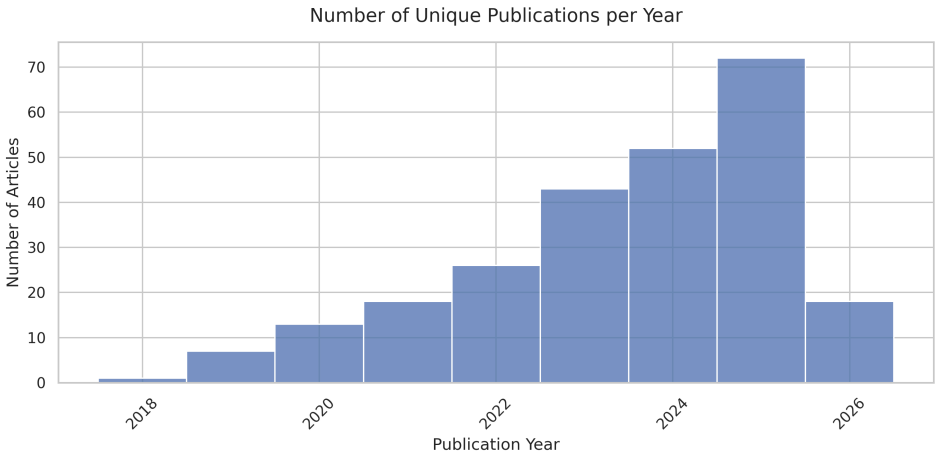

Figure 2: Distribution of unique publications by year.

3.4.3 Top Publication Sources

An analysis of the publication venues (Figure 3) reveals that a significant portion of research in this domain is published across IEEE conferences and specialized computer networks journals.

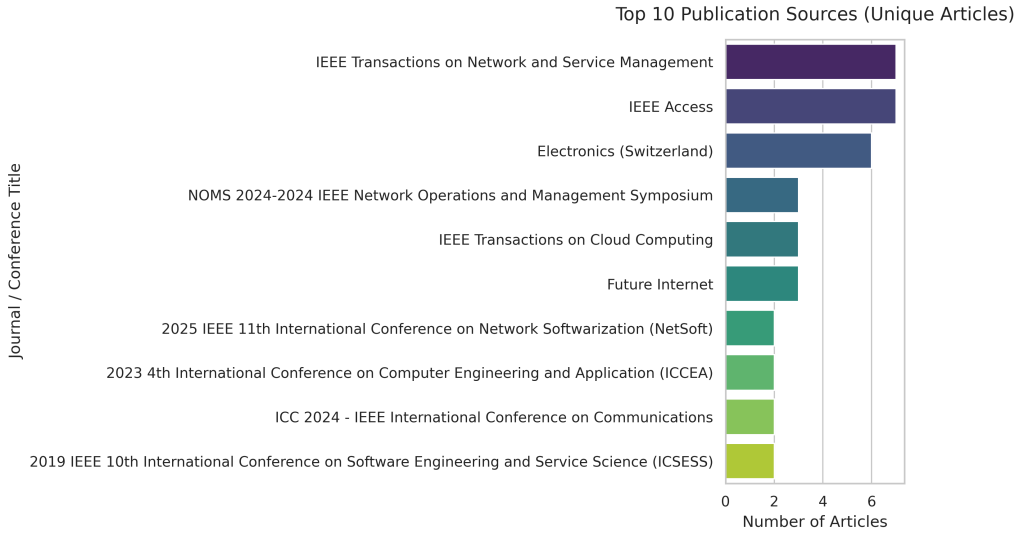

Figure 3: Top 10 publication sources for the filtered articles.

4 Conclusions

4.1 SLR process

Conducting this systematic literature review revealed that while “service mesh” is a highly popular industry topic, rigorous academic benchmarking remains relatively niche. The initial search across IEEE Xplore, Scopus, arXiv, and SpringerLink yielded 287 articles. A major challenge during the screening process was the prevalence of general cloud-computing or microservices papers that only mentioned service meshes in passing without providing quantitative evaluations. Consequently, careful manual screening was required to isolate the final 12 articles that actually provided empirical performance data. The process demonstrated the necessity of highly specific search strings and strict quality criteria to filter out industry buzzwords and focus on true architectural benchmarking.

4.2 SLR results

Based on the initial data extracted from the core publications, several distinct themes have emerged regarding the impact of service mesh solutions on microservice efficiency:

1. **The Cost of Security (mTLS) and Sidecar Proxies:** Deploying a traditional sidecar architecture introduces measurable latency and memory/CPU overhead [6]. This is largely driven by the cryptographic costs of mutual TLS and the underlying Linux network stack (e.g., Iptables rule matching), which becomes a significant bottleneck, particularly in latency-sensitive edge computing environments [8].
2. **Linkerd vs. Istio Performance:** Direct experimental comparisons highlight that Linkerd generally offers a lighter footprint with lower average latency (e.g., 104 ms vs. 135 ms) and better stability under high load compared to Istio [11]. However, Istio remains the standard for environments requiring highly complex traffic routing and granular observability configurations.
3. **Architectural Evolution (eBPF and Sidecar-less):** To mitigate sidecar overheads, the research landscape is shifting toward kernel-level optimizations. Newer architectures utilizing eBPF are showing immense promise in drastically reducing network latency while maintaining robust security and observability, effectively bypassing traditional proxy bottlenecks [3], [6].
4. **Trade-offs Between Efficiency and Resilience:** While service meshes impose a performance penalty, they significantly enhance system stability [5]. Studies utilizing chaos engineering indicate that environments equipped with Istio recover faster and handle failure injections much more gracefully than native Kubernetes deployments, proving that the performance trade-off is often justified by massive gains in fault tolerance [2], [7].

These findings directly validate the necessity of our research project. The literature confirms that service mesh performance overhead is highly contextual, reinforcing our goal to experimentally test these solutions in a controlled environment to find the optimal balance between security, observability, and efficiency.

5 Literature

Bibliography

[1] M. Nagy, T. Lévai, F. Németh, A. Panda, G. Antichi, and G. Rétvári, "Elastic Scaling of Real-Time Communication Services," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 3393–3405, 2026, doi: 10.1109/TNSM.2026.3674598.

[2] I. Dermentzis, G. Koukis, and V. Tsaoussidis, "Trade-Offs in Kubernetes Security and Energy Consumption," *Urban Science*, vol. 10, no. 2, p. 81, Feb. 2026, doi: 10.3390/urbansci10020081.

[3] W. Yang, P. Chen, G. Yu, H. Zhang, and H. Zhang, "Network shortcut in data plane of service mesh with eBPF," *Journal of Network and Computer Applications*, vol. 222, p. 103805, Feb. 2024, doi: 10.1016/j.jnca.2023.103805.

[4] H.-Y. Huang, Y.-Y. Fanjiang, C.-H. Hung, H.-Y. Tsai, and B.-H. Lin, "Evaluation of a Smart Intercom Microservice System Based on the Cloud of Things," *Electronics*, vol. 12, no. 11, p. 2406, May 2023, doi: 10.3390/electronics12112406.

[5] S. Singh, C. H. Muntean, and S. Gupta, "Resilient microservices: an investigation into Istio effectiveness in Kubernetes," *Cluster Computing*, vol. 29, no. 1, p. 27, Feb. 2026, doi: 10.1007/s10586-025-05750-x.

[6] A. B. Barr, O. Lavi, Y. Naor, S. Rampal, and J. Tavori, "Technical Report: Performance Comparison of Service Mesh Frameworks: the MTLS Test Case." Accessed: Apr. 05, 2026. [Online]. Available: https://arxiv.org/abs/2411.02267v1

[7] L. Larsson, W. Tärneberg, C. Klein, E. Elmroth, and M. Kihl, “Impact of etcd Deployment on Kubernetes, Istio, and Application Performance.” Accessed: Apr. 05, 2026. [Online]. Available: https://arxiv.org/abs/2004.00372v1

[8] M. Ganguli, S. Ranganath, S. Ravisundar, A. Layek, D. Ilangovan, and E. Verplanke, “Challenges and Opportunities in Performance Benchmarking of Service Mesh for the Edge,” in *2021 IEEE International Conference on Edge Computing (EDGE)*, Chicago, IL, USA: IEEE, Sept. 2021, pp. 78–85. doi: 10.1109/EDGE53862.2021.00020.

[9] D. A. Hahn, D. Davidson, and A. G. Bardas, “Security Issues and Challenges in Service Meshes – An Extended Study.” Accessed: Apr. 05, 2026. [Online]. Available: https://arxiv.org/abs/2010.11079v1

[10] Q. Zeng, M. Kavousi, Y. Luo, L. Jin, and Y. Chen, “Full-stack vulnerability analysis of the cloud-native platform,” *Computers & Security*, vol. 129, p. 103173, June 2023, doi: 10.1016/j.cose.2023.103173.

[11] K. Bosquez, X. Caiza, L. Núñez, and J. Monar, “Comparative Evaluation of Linkerd and Istio Service Meshes in a Microservices Architecture Application,” in *2025 IEEE Colombian Caribbean Conference (C3)*, Santa Marta, Colombia: IEEE, Sept. 2025, pp. 1–6. doi: 10.1109/C366505.2025.11340184.

[12] S. Kurpad, S. Bt, S. Vijaykumar, S. Jain, and S. Kalambur, “Microarchitectural Analysis and Characterization of Performance Overheads in Service Meshes with Kubernetes,” in *2023 3rd Asian Conference on Innovation in Technology (ASIANCON)*, Ravet IN, India: IEEE, Aug. 2023, pp. 1–6. doi: 10.1109/ASIANCON58793.2023.10270428.